Práctica 3 - Representación de la información

Sistemas Digitales

Segundo Cuatrimestre 2026

Ejercicio 1.

1. Convierta 13, 42, 127, 128 y 255 de decimal a binario. Use la mínima cantidad de bits y luego escriba cada valor con 8 bits cuando sea posible.
2. Convierta a decimal los numerales 101101_2, 11111111_2, $2A_{16}$ y CAFE_{16}.
3. Convierta `16'b1010_1110_0011_0101` a hexadecimal agrupando bits, sin pasar por decimal.
4. Convierta `16'h7B2D` a binario, conservando exactamente 16 bits.
5. Explique por qué `8'b0010_1010` y `6'b10_1010` representan el mismo entero sin signo pero no tienen la misma precisión.
6. Determine la mínima cantidad de bits necesaria para representar, sin signo, cada uno de estos rangos: 0 a 99; 0 a 1000; 0 a 1023; 0 a 1024.

Ejercicio 2.

Considere enteros sin signo de N bits.

1. Escriba el rango general y los rangos para $N = 4$, 8 y 12.
2. Extienda `4'b1011` a 8 y 12 bits preservando su valor sin signo.
3. Trunque `8'b0010_1101` y `8'b1100_0101` a 6 bits. Interprete antes y después como sin signo e indique en qué caso se conserva el valor.
4. Formule una condición sobre los bits descartados que garantice que un truncamiento sin signo conserva el valor.
5. Un sensor produce valores entre 0 y 750 inclusive. Determine el ancho mínimo de su salida. Si se reserva además un patrón para indicar "medición inválida", determine si ese ancho sigue siendo suficiente.

Ejercicio 3.

1. Demuestre que el mayor entero sin signo representable con N bits es $2^N - 1$.
2. Demuestre que la suma de dos números de N bits produce a lo sumo un carry de salida de un bit.
3. Al agregar un bit a una representación sin signo, ¿cuántos valores nuevos se vuelven representables? Exprese la respuesta en función de N y explique su relación con duplicar el espacio de códigos.

Ejercicio 4.

Complete la tabla interpretando cada cadena primero como sin signo y luego como complemento a dos. Todas tienen 8 bits.

Cadena	sin signo	Complemento a dos
`8'b0000_0000`		
`8'b0000_0001`		
`8'b0111_1111`		
`8'b1000_0000`		
`8'b1011_1111`		

Cadena	sin signo	Complemento a dos
`8'b1111_1111`		

1. Identifique mínimo y máximo en cada interpretación.
2. ¿Qué cadenas representan el mismo valor bajo ambas interpretaciones? Generalice la respuesta para N bits.
3. Compare brevemente complemento a dos con signo y magnitud para las cadenas `8'b1000_0000` y `8'b1111_1111`. ¿Qué inconvenientes de signo y magnitud aparecen en esos ejemplos?

Ejercicio 5.

1. Represente 0, 1, 5, 7, -1, -3, -7 y -8 en complemento a dos de 4 bits. Marque los que no sean representables.
2. Represente 63, 127, -64, -127 y -128 en complemento a dos de 8 bits. Marque los que no sean representables.
3. Para `8'b0010_1101`, `8'b1011_1111` y `8'b0000_0000`, invierta todos los bits y sume uno conservando 8 bits. Interprete el resultado y compruebe que representa el inverso aditivo.
4. Aplique el mismo método a `8'b1000_0000`. ¿Qué resultado obtiene? Explique por qué no contradice el método y qué propiedad del rango revela.

Ejercicio 6.

Para cada patrón de 4 bits, interprételo en complemento a dos, extiéndalo a 8 bits conservando el valor y compruebe el resultado: `4'b0011`, `4'b0111`, `4'b1000`, `4'b1011` y `4'b1111`.

1. Compare la extensión de signo de `4'b1011` con su extensión con ceros. Interprete ambas cadenas resultantes como complemento a dos.
2. Trunque a 4 bits `8'b0000_0101`, `8'b1111_1011`, `8'b0001_0101` y `8'b1110_1011`. Indique en cuáles se conserva el valor signado.
3. Proponga una condición sobre los bits descartados y el nuevo bit de signo que permita decidir si un truncamiento conserva un valor en complemento a dos.
4. Explique por qué “copiar el bit de signo” no es una operación especial sobre números, sino una forma concreta de cablear los bits agregados.

Ejercicio 7.

Realice cada suma directamente en binario, conservando 6 bits. Indique el carry de salida. Luego interprete operandos y resultado como (a) sin signo y (b) complemento a dos, e indique si la operación matemática cabe en cada interpretación.

Caso	Operando A	Operando B
A	`6'b100001`	`6'b011110`
B	`6'b100001`	`6'b011111`
C	`6'b001111`	`6'b001111`
D	`6'b111101`	`6'b111110`
E	`6'b010100`	`6'b011001`

1. Para cada caso registre: resultado de 6 bits, carry, resultado sin signo, resultado signado y existencia de overflow signado.
2. Encuentre un caso con carry pero sin overflow y otro con overflow pero sin carry.
3. Explique por qué el circuito que suma los bits puede ser el mismo aunque la validez del resultado dependa de la interpretación.

Ejercicio 8.
Trabaje en complemento a dos de 4 bits, cuyo rango es $[-8, 7]$. Encuentre un par de operandos representables para cada condición, o demuestre que no existe:

1. sin carry y sin overflow;
2. con carry y sin overflow;
3. con carry y con overflow;
4. sin carry y con overflow;
5. resultado almacenado cero con carry;
6. resultado almacenado cero sin carry;
7. resultado almacenado negativo con overflow;
8. resultado almacenado negativo sin overflow.

En cada caso escriba los operandos y el resultado tanto en binario como en decimal. Finalmente, enuncie una regla para detectar overflow observando los signos de los operandos y del resultado.

Ejercicio 9.
Calcule las siguientes restas con 8 bits implementando $A - B$ como $A + \neg(B)$. Donde en este caso el operando $(\neg)$ es el complemento aditivo (swap bit a bit y luego sumar 1). Para cada una, escriba el resultado almacenado, interprételo en decimal e indique si ocurre overflow signado:

1. $37 - 12$;
2. $12 - 37$;
3. $100 - (-20)$;
4. $-100 - 20$;
5. $-128 - (-1)$;
6. $127 - (-1)$.

Derive una regla de overflow para la resta en términos de los signos de A, B y el resultado. Explique por qué el carry de salida no puede usarse como sinónimo de overflow.

Ejercicio 10.
Se deben sumar los siguientes valores, todos codificados en complemento a dos de 16 bits:

$$7744_{16} + 5499_{16} + 6788_{16} + \text{AB68}_{16} + \text{88BD}_{16} + 9879_{16}$$

1. Interprete cada operando como entero con signo y verifique que el resultado matemático final es representable con 16 bits.
2. Evalúe la expresión de izquierda a derecha conservando 16 bits después de cada suma. Identifique los pasos que producen overflow.
3. Reordene los operandos para que ninguna suma parcial produzca overflow.
4. Resuelva nuevamente usando acumulador de 18 bits con extensión de signo. ¿Sigue importando el orden para obtener el resultado matemático correcto?
5. Explique por qué la asociatividad de la suma matemática no garantiza que cualquier orden sea seguro bajo precisión fija.

Ejercicio 11.
Escriba un módulo puramente combinacional que reciba `logic [7:0] x` y `logic [7:0] y` y produzca:

- `sum`, los 8 bits menos significativos de la suma;
- `carry_sin signo`, el carry de salida;
- `overflow_signado`, overflow al interpretar ambos operandos en complemento a dos;
- `x_extended_sin signo[15:0]`, extensión con ceros de `x`
- `x_extended_signado[15:0]`, extensión de signo de `x`.

1. Antes de programar, escriba ecuaciones booleanas para los dos flags.
2. Implemente el módulo sin instanciar todavía sumadores a nivel de compuertas. Use un cálculo de 9 bits para no perder el carry.
3. Construya un testbench autocontrolado que incluya, al menos, estos pares hexadecimales: `(01, 01)`, `(7F, 01)`, `(FF, 01)`, `(80, 80)`, `(80, FF)` y `(40, C0)`.
4. Para cada caso, muestre los operandos y el resultado en hexadecimal, decimal sin signo y decimal signado. Calcule el valor esperado con una expresión de mayor ancho y emita `$error` ante diferencias.
5. Explique qué salidas dependen solamente del patrón de bits y cuáles expresan una interpretación del patrón.
6. Verifique explícitamente que no se usó estado, reloj ni `always_ff`: este ejercicio estudia significado y operaciones, no construye aún la ALU modular de la unidad siguiente.